DEPARTMENT OF COMPUTER SCIENCE & INFORMATION TECHNOLOGY

OBJECT ORIENTED PROGRAMMING

Course Code: CT-260

ASSIGNMENT — DESIGN PATTERNS IN REAL-WORLD SOFTWARE SYSTEMS

Semester	Spring 2026
Class	1st Year — BS Computer Science & Information Technology
Instructor	Dr. Murk Marvi
Total Marks	5 Marks
Submission Deadline	30 Days
Assignment Type	Group Assignment, 3 members in a group

PROJECT :

File Encryption Suite with Multiple Ciphers (CipherCascade)

GROUP MEMBERS :

Minahil Khan CT-25066

Rubaisha Arif CT-25067

Task A — Application Overview

What does the application do?

CipherCascade is a C++ file encryption and decryption suite inspired by CyberChef — a browser-based data transformation tool developed by GCHQ. Just as CyberChef allows users to chain together multiple operations (Base64, AES, XOR, and 300+ others) into a sequential recipe that transforms data step by step, CipherCascade implements the same pipeline philosophy in C++. The user selects up to three cipher algorithms that are applied one after another, where the output of each cipher becomes the input of the next, producing a multi-layered encrypted file. Decryption works by applying the same ciphers in reverse order with the same keys. The application works on any binary file — plain text, images, or raw data.

Who are its users?

CipherCascade targets students learning cryptography and object-oriented programming, software developers experimenting with classical cipher algorithms, and security enthusiasts who want to understand how chained encryption pipelines work. It is also useful as an educational demonstration tool for instructors teaching design patterns, since the project clearly illustrates Strategy, Chain of Responsibility, and Facade patterns in a real working application.

What are its core features and modules?

- User Entry / Mode Selection — the user provides the file path and chooses encrypt or decrypt mode at startup, acting as the entry point to the system
- Cipher Catalogue (4 algorithms) — Vigenere Cipher (string key, shifts alphabetic characters), XOR Cipher (integer key, bitwise XOR on every byte), Byte Reversal Cipher (reverses entire byte sequence, no key), Atbash Cipher (maps each byte using 255 - byte, no key)
- 3-Step Pipeline (Chain) — the user selects three ciphers in sequence; each cipher's output file feeds the next as input, mirroring CyberChef's chainable Bake pipeline
- File I/O Module — the Ciphers base class handles binary file reading into a vector<char> buffer and writing the result to EncryptedVer.enc or DecryptedVer
- Output / Notifications — the console prints confirmation messages after each step (e.g. 'Encrypted file saved as EncryptedVer.enc'), keeping the user informed of progress

Why did you choose this application?

We chose to model CipherCascade on CyberChef because it is a real, widely used security tool that perfectly demonstrates the power of chaining multiple operations. CyberChef's recipe-based design maps naturally onto three major OOP design patterns: Strategy (each operation is an interchangeable algorithm), Chain of Responsibility (output of one step feeds the next), and Facade (the user sees a simple menu while complex binary I/O is hidden). Implementing a simplified version of this concept in C++ allowed us to apply theoretical design pattern knowledge to a practical, working cryptography application, making the assignment both academically meaningful and technically interesting.

Task B — Design Pattern Identification

Strategy Pattern	Chain of Responsibility	Facade Pattern

Pattern 1 — Strategy (Behavioral)

Where does it appear?

In our system, the abstract base class Ciphers defines the common Strategy interface:

```
virtual void encrypt() = 0;
virtual void decrypt() = 0;
```

The derived classes each provide their own concrete implementation of these algorithms:

- VigenereCipher — shifts alphabetic bytes using a repeating string key
- XORCipher — XOR-masks every byte with an integer key
- ByteReversalCipher — reverses the entire byte sequence of the file
- AtbashCipher — maps each byte using the formula 255 − byte

Why Strategy?

If all encryption algorithms were written inside one large class using if-else or switch statements, the code would become difficult to manage and extend. The Strategy pattern allows each encryption algorithm to be implemented in a separate class while sharing a common interface. This makes the algorithms interchangeable and easy to swap at runtime without modifying existing code.

OOP Principles Reinforced

- Polymorphism — same interface, different runtime behaviour
- Abstraction — users call encrypt()/decrypt() without knowing internals
- Open/Closed Principle — new ciphers added without editing existing classes

Pattern 2 — Chain of Responsibility (Behavioral)

Where does it appear?

In the main() function, the program allows the user to select up to three ciphers sequentially. After one cipher processes the file, the output file (EncryptedVer.enc) becomes the input path for the next selected cipher. The file therefore passes through multiple cipher handlers one after another.

Why Chain of Responsibility?

Each cipher acts as an independent handler that processes the file and then forwards responsibility to the next cipher by updating the file path variable. This allows multi-layer encryption without any tight

coupling between the cipher classes. The user controls the order of execution, creating a flexible and composable encryption chain — analogous to CyberChef's Bake pipeline.

OOP Principles Reinforced

- Modularity — each cipher is a self-contained processing unit
- Separation of Responsibilities — each class has one main job
- Loose Coupling — cipher classes are not directly dependent on one another

Pattern 3 — Facade (Structural)

Where does it appear?

The Ciphers base class constructor handles all binary file I/O (reading the file into a vector<char> buffer using ios::binary, seekg, tellg, and file.read). The saveEncrypted() and saveDecrypted() methods handle writing output. The main() function wraps everything in a simple numbered menu. All of this complexity is invisible to the end user.

Why Facade?

The Facade pattern provides a simplified interface over a complex subsystem. Without this pattern, users would need to understand binary I/O, memory buffers, pointer arithmetic, and cipher mathematics directly. Instead, the Ciphers class and main() act as a Facade: the user only sees a menu with 1–4 choices, a filename prompt, and an encrypt/decrypt toggle. Everything else is hidden.

OOP Principles Reinforced

- Encapsulation — internal complexity is hidden from the client
- Abstraction — the menu is the only user-facing interface
- Code Reusability — file I/O code written once in the base class, reused by all ciphers

Task C — C++ Implementation

Below is the complete C++ implementation of CipherCascade, demonstrating all three design patterns.

Base Class — Ciphers

```
#include <iostream>
#include <vector>
#include <fstream>
#include <string>
#include <cstdlib>
#include <algorithm>
using namespace std;

class Ciphers {
private:
    string filename;
protected:
    vector<char> holder;
    int size;
public:
    virtual void encrypt() = 0;
    virtual void decrypt() = 0;

    void saveEncrypted() {
        ofstream outfile("EncryptedVer.enc", ios::binary);
        outfile.write(holder.data(), holder.size());
        outfile.close();
        cout << "Encrypted file saved as EncryptedVer.enc" << endl;
    }

    void saveDecrypted() {
        ofstream outfile("DecryptedVer", ios::binary);
        outfile.write(holder.data(), holder.size());
        outfile.close();
        cout << "Decrypted file saved as DecryptedVer" << endl;
    }

    Ciphers(string n) : filename(n) {
        ifstream file(n, ios::binary);
        if (!file.is_open()) { cout << "Error: " << n; exit(1); }
        file.seekg(0, ios::end);
        size = file.tellg();
        file.seekg(0, ios::beg);
        holder.resize(size);
        file.read(holder.data(), size);
```

```
        file.close();
        cout << "Read " << size << " bytes into memory" << endl;
    }

    string getFilename() const { return filename; }
    int getsize() const { return size; }
    virtual ~Ciphers() {}
};
```

Derived Cipher Classes

```
//1st Cipher
// Atbash Cipher class
class AtbashCipher : public Ciphers {
public:
    AtbashCipher(string n) : Ciphers(n) {}
    void encrypt() override {
        for (int i = 0; i < holder.size(); i++) {
            holder[i] = 255 - (unsigned char)holder[i];
        }
    }
    void decrypt() override {
        for (int i = 0; i < holder.size(); i++) {
            holder[i] = 255 - (unsigned char)holder[i];
        }
    }
};

//2nd Cipher
// Byte Reversal Cipher class
class ByteReversalCipher : public Ciphers {
public:
    ByteReversalCipher(string n) : Ciphers(n) {}
    void encrypt() override {
        reverse(holder.begin(), holder.end());
    }
    void decrypt() override {
        reverse(holder.begin(), holder.end()); // Reverse back
    }
};

//3rd Cipher
// Vigenere Cipher class
class VigenereCipher : public Ciphers {
private:
    string key;
```

```
public:
    VigenereCipher(string n, string k) : Ciphers(n) {
        key = k;
    }
    void encrypt() override {
        int keyIndex = 0; // Key index
        for (int i = 0; i < holder.size(); i++) {
            char c = holder[i];
            if (isalpha(c)) {
                char k = key[keyIndex % key.length()];
                int kValue = toupper(k) - 'A';
                if (isupper(c)) {
                    int tValue = c - 'A';
                    holder[i] = ((tValue + kValue) % 26) + 'A';
                }
                else { // Lowercase
                    int tValue = c - 'a';
                    holder[i] = ((tValue + kValue) % 26) + 'a';
                }
                keyIndex++; // Move to next key character
            }
        }
    }
    void decrypt() override {
        int keyIndex = 0;
        for (int i = 0; i < holder.size(); i++) {
            char c = holder[i];
            if (isalpha(c)) {
                char k = key[keyIndex % key.length()];
                int kValue = toupper(k) - 'A';
                if (isupper(c)) {
                    int cValue = c - 'A';
                    holder[i] = ((cValue - kValue + 26) % 26) + 'A';
                }
                else { // Lowercase
                    int cValue = c - 'a';
                    holder[i] = ((cValue - kValue + 26) % 26) + 'a';
                }
                keyIndex++; // Move to next key character
            }
        }
    }
};

//4th Cipher
// XOR Cipher class
class XORCipher : public Ciphers {
```

```
private:
    int key;
public:
    XORCipher(string n, int k) : Ciphers(n) {
        key = k;
    }
    void encrypt() override {
        for (int i = 0; i < holder.size(); i++) {
            holder[i] = holder[i] ^ key;
        }
    }
    void decrypt() override {
        for (int i = 0; i < holder.size(); i++) {
            holder[i] = holder[i] ^ key; // XOR reverses itself
        }
    }
};

// Main program
int main() {
    cout << "\n";
    cout << "==================================\n";
    cout << " C I P H E R C A S C A D E \n";
    cout << "==================================\n";
    cout << " Vigenere XOR Byte Rev \n";
    cout << " Atbash \n";
    cout << " 3-Step Chain \n";
    cout << "==================================\n\n";

    int n = 1;
    int choice;
    string file, key;
    int s;

    cout << "Enter the path/file you want to encrypt/decrypt: " << endl;
    cin >> file;
    cout << "Do you want to encrypt the file or decrypt it?" << endl;
    cout << "1. Encrypt" << endl << "2. Decrypt" << endl;
    cin >> s;

    while (n <= 3) {
        cout << "Select " << n << " cipher:" << endl;
        cout << "1. Vigenere Cipher" << endl;
        cout << "2. XOR Cipher" << endl;
        cout << "3. Byte Reversal Cipher" << endl;
        cout << "4. Atbash Cipher" << endl;
        cin >> choice;
```

```
        switch (choice) {
        //case 1
        case 1: {
            cout << "Enter string key: " << endl;
            cin >> key;
            VigenereCipher v1(file, key);
            if (s == 1) {
                cout << "Encrypting file through Vigenere Cipher..." << endl;
                v1.encrypt();
                v1.saveEncrypted();
                file = "EncryptedVer.enc";
            } else {
                cout << "Decrypting file through Vigenere Cipher..." << endl;
                v1.decrypt();
                v1.saveDecrypted();
                file = "DecryptedVer";
            }
            break;
        }
        //case 2
        case 2: {
            int k;
            cout << "Enter integer key: " << endl;
            cin >> k;
            XORCipher x1(file, k);
            if (s == 1) {
                cout << "Encrypting file through XOR Cipher..." << endl;
                x1.encrypt();
                x1.saveEncrypted();
                file = "EncryptedVer.enc";
            } else {
                cout << "Decrypting file through XOR Cipher..." << endl;
                x1.decrypt();
                x1.saveDecrypted();
                file = "DecryptedVer";
            }
            break;
        }
        //case 3
        case 3: {
            ByteReversalCipher b1(file);
            if (s == 1) {
                cout << "Encrypting file through Byte Reversal Cipher..." <<
endl;
                b1.encrypt();
                b1.saveEncrypted();
                file = "EncryptedVer.enc";
```

```
            } else {
                cout << "Decrypting file through Byte Reversal Cipher..." <<
endl;
                b1.decrypt();
                b1.saveDecrypted();
                file = "DecryptedVer";
            }
            break;
        }
        //case 4
        case 4: {
            AtbashCipher a1(file);
            if (s == 1) {
                cout << "Encrypting file through Atbash Cipher..." << endl;
                a1.encrypt();
                a1.saveEncrypted();
                file = "EncryptedVer.enc";
            } else {
                cout << "Decrypting file through Atbash Cipher..." << endl;
                a1.decrypt();
                a1.saveDecrypted();
                file = "DecryptedVer";
            }
            break;
        }
        default:
            cout << "Error: incorrect selection;" << endl;
            break;
        }
        n++;
    }
}
```

Outputs

Encryption Output

The following shows a 3-step encryption run on test.txt — XOR (key: 6) → Byte Reversal → Vigenere (key: ITSKEY):

```
===================================
 C I P H E R C A S C A D E
===================================
 Vigenere  XOR  Byte Rev
           Atbash
        3-Step Chain
===================================

Enter the path/file you want to encrypt/decrypt:
test.txt
Do you want to encrypt the file or decrypt it?
1. Encrypt
2. Decrypt
1
Select 1 cipher:
1.Vigenere Cipher  2.Xor Cipher  3.Byte Reversal Cipher  4.Atbash Cipher
2
Enter (int) key:
6
Successfully read 454 bytes into memory
Encrypting file through XOR Cipher...
Encrypted file saved as EncryptedVer.enc

Select 2 cipher:
1.Vigenere Cipher  2.Xor Cipher  3.Byte Reversal Cipher  4.Atbash Cipher
3
Successfully read 454 bytes into memory
Encrypting file through Byte Reversal Cipher...
Encrypted file saved as EncryptedVer.enc

Select 3 cipher:
1.Vigenere Cipher  2.Xor Cipher  3.Byte Reversal Cipher  4.Atbash Cipher
1
Enter string key:
ITSKEY
Successfully read 454 bytes into memory
Encrypting file through Vigenere Cipher...
Encrypted file saved as EncryptedVer.enc

Process exited after 226 seconds with return value 0
Press any key to continue . . .
```

Decryption Output

Decryption is performed in reverse order using the same keys — Vigenere (key: ITSKEY) → Byte Reversal → XOR (key: 6):

```
===================================
```

```
 C I P H E R C A S C A D E
==================================
 Vigenere  XOR  Byte Rev
           Atbash
        3-Step Chain
==================================

Enter the path/file you want to encrypt/decrypt:
EncryptedVer.enc
Do you want to encrypt the file or decrypt it?
1. Encrypt
2. Decrypt
2
Select 1 cipher:
1.Vigenere Cipher  2.Xor Cipher  3.Byte Reversal Cipher  4.Atbash Cipher
1
Enter string key:
ITSKEY
Successfully read 454 bytes into memory
Decrypting file through Vigenere Cipher...
Decrypted file saved as DecryptedVer

Select 2 cipher:
1.Vigenere Cipher  2.Xor Cipher  3.Byte Reversal Cipher  4.Atbash Cipher
3
Successfully read 454 bytes into memory
Decrypting file through Byte Reversal Cipher...
Decrypted file saved as DecryptedVer

Select 3 cipher:
1.Vigenere Cipher  2.Xor Cipher  3.Byte Reversal Cipher  4.Atbash Cipher
2
Enter (int) key:
6
Successfully read 454 bytes into memory
Decrypting file through XOR Cipher...
Decrypted file saved as DecryptedVer

Process exited after 37.28 seconds with return value 0
Press any key to continue . . .
```

File Verification — Original vs Encrypted vs Decrypted

The table below shows the content state of test.txt at each stage of the pipeline:

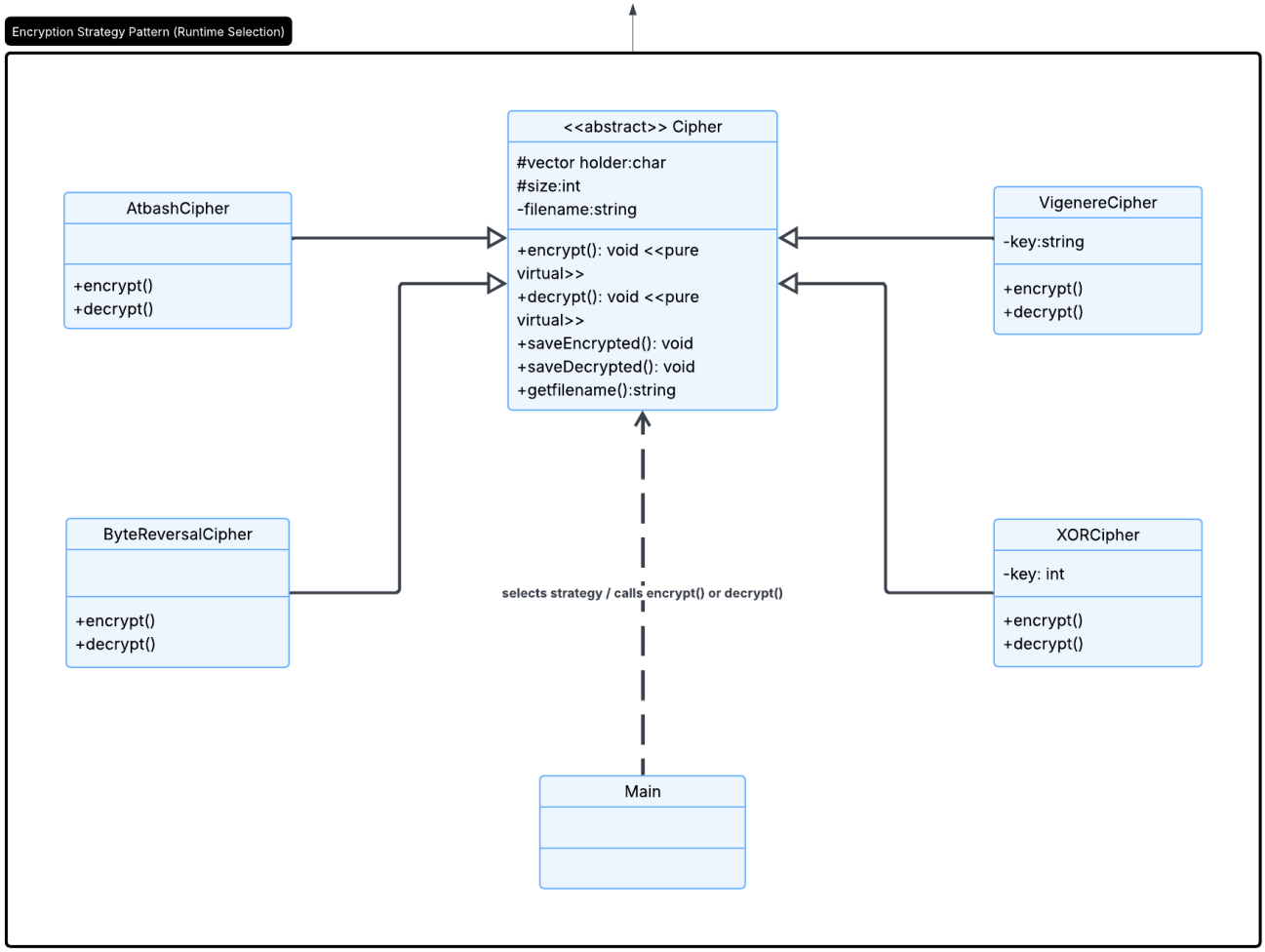
Original (test.txt)	Encrypted (EncryptedVer.enc)	Decrypted (DecryptedVer)
HELLO WORLD! This is my encryption test Honesty is the practice of speaking and acting truthfully...	(cmuxvg&vkga&yfq#jokrkkzy&xqq#ag &rvmzxurlgm&uzq&lvmh jrglzly&... *ctzynak*&uzq&ypvtksrp#bla&bwglz 'FHBBI&SNMKG *[binary / unreadable data]*	HELLO WORLD! This is my encryption test Honesty is the practice of speaking and acting truthfully...

Task D — UML Class Diagrams

Four UML diagrams are provided: one per design pattern (Strategy, Chain of Responsibility, Facade), followed by the complete overall class diagram of the entire CipherCascade system.

Pattern 1 — Strategy Pattern: UML Class Diagram

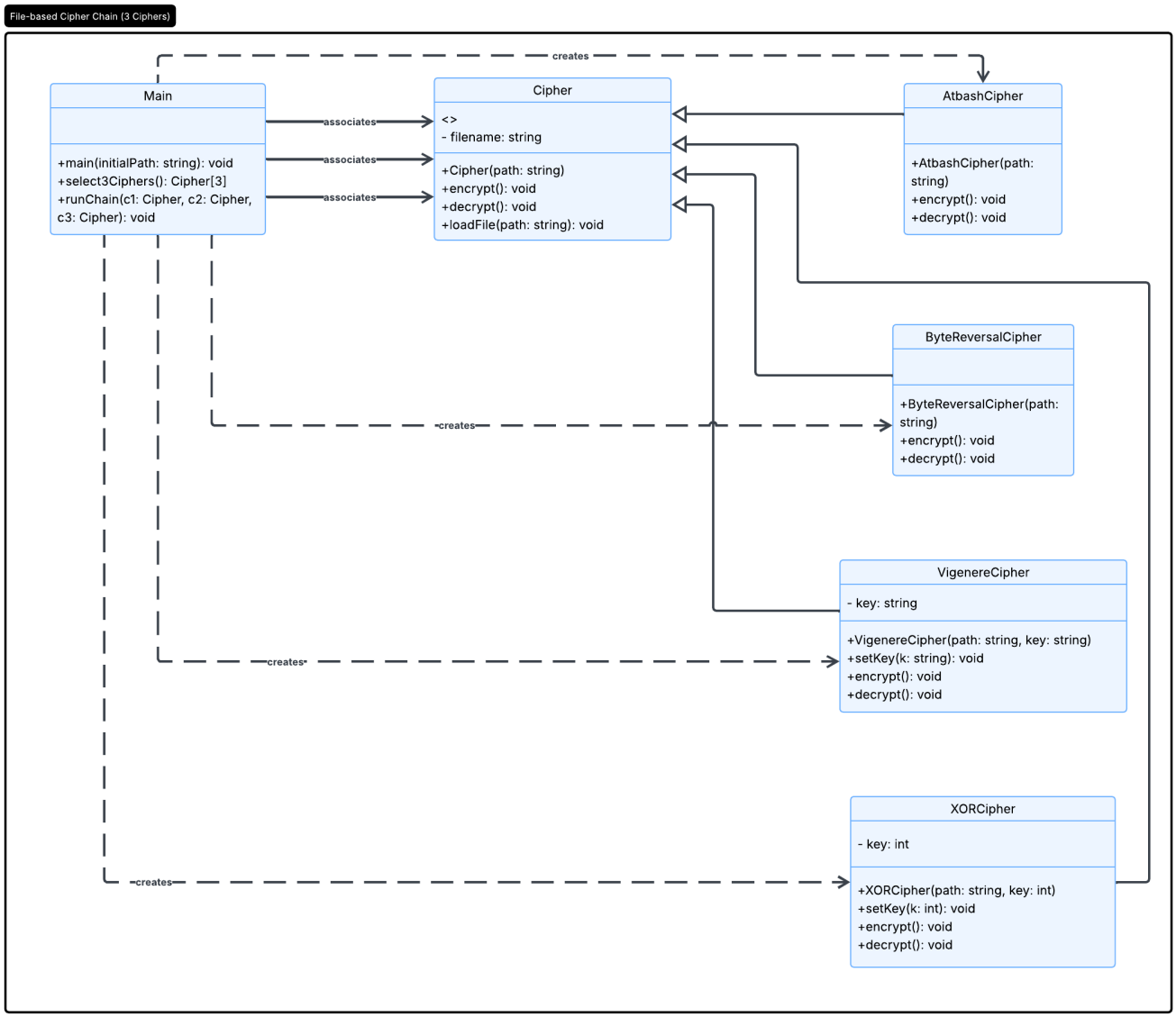
Relationship: Generalisation (Inheritance) — open hollow triangle arrow (——▷) from each ConcreteStrategy to abstract Ciphers



This UML shows the **Strategy pattern**, where **main()** acts as the context and selects an encryption or decryption algorithm at runtime through the abstract **Cipher** interface. The concrete classes **AtbashCipher**, **ByteReversalCipher**, **VigenereCipher**, and **XORCipher** all inherit from **Cipher**, so the client can switch algorithms without changing how it calls encrypt or decrypt.

Pattern 2 — Chain of Responsibility: UML Class Diagram

Relationship: Generalisation (——▷) from each ConcreteHandler to Ciphers. Object Flow (—>) shows file passing through each handler in sequence.

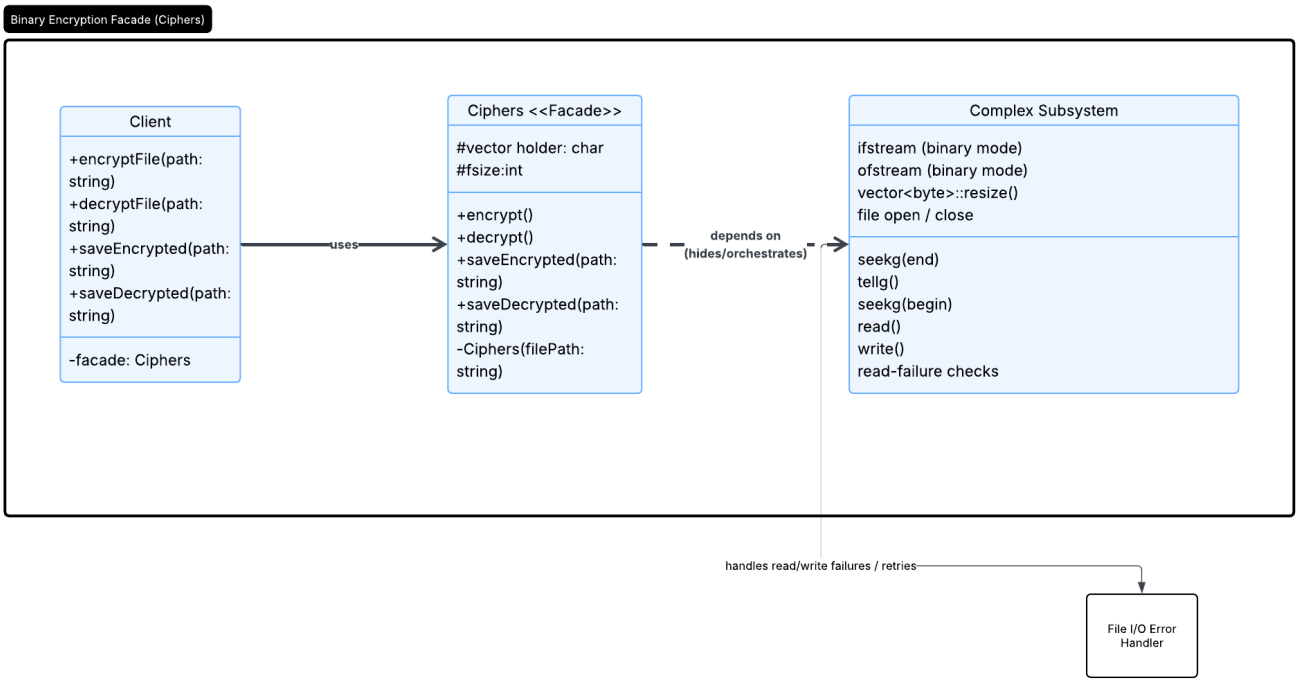


Object / Data Flow — file path passed through the chain:

Each ConcreteHandler (cipher) processes the file and saves the output. The file path variable is then updated in main() so the next handler reads the previous output — modelling the chain of responsibility without any direct coupling between cipher classes.

Pattern 3 — Facade Pattern: UML Class Diagram

Relationship: Dependency (⋯>) from Client to Facade; Dependency (⋯>) from Facade to each Subsystem class.

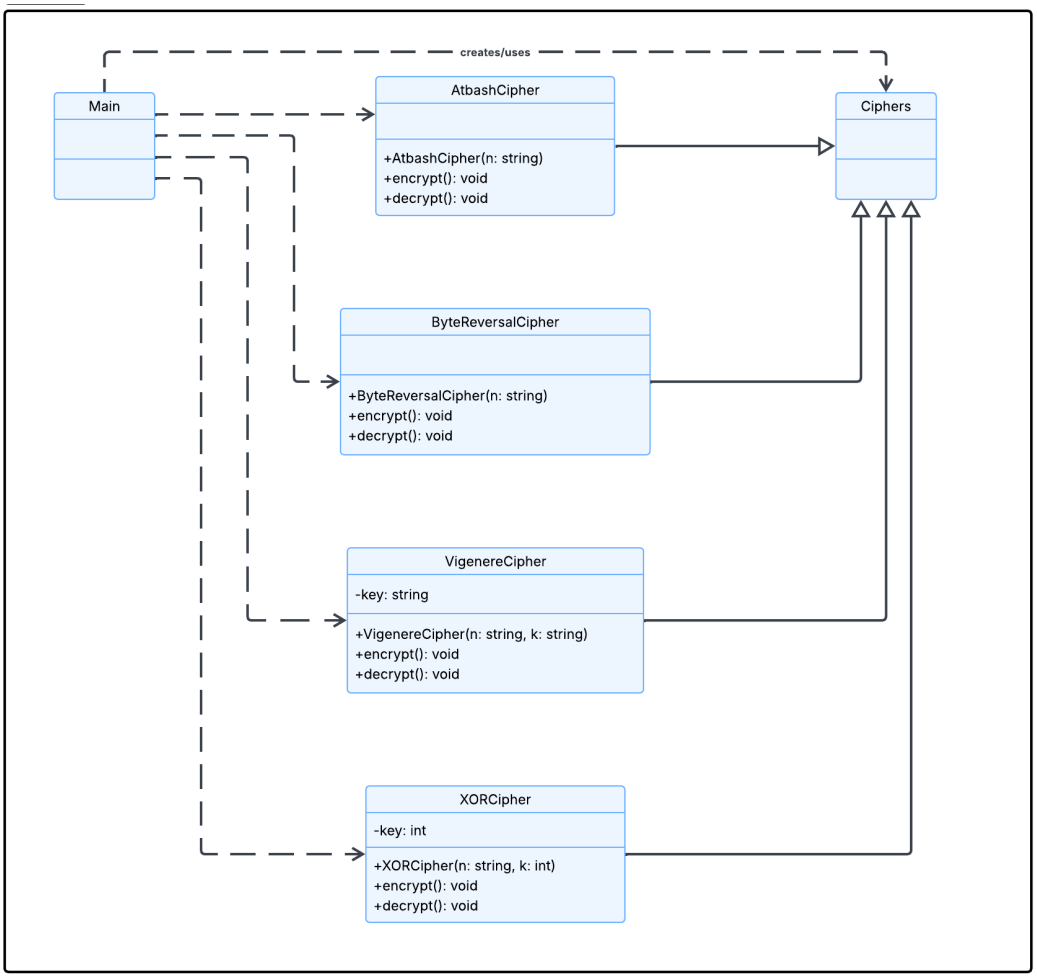


This UML shows the **Facade pattern** where the **Client** interacts only with the **Ciphers** class through simple methods like encrypt, decrypt, and save operations. The **Ciphers** facade hides the low-level binary file handling details by coordinating the **Complex Subsystem**, which performs file opening, seeking, reading, writing, and buffer management.

Complete System UML Class Diagram

This diagram shows the full CipherCascade class hierarchy with all class names, attributes (with visibility and types), methods (with signatures), and UML relationships.

- private Visibility	# protected Visibility	+ public Visibility	—▷ Generalisation Relationship



UML Relationships Used	Design Pattern Mapping
—▷ Generalisation: all 4 subclasses inherit from Ciphers - private / # protected / + public {abstract} marks pure virtual methods	Strategy: Ciphers = interface; subclasses = strategies Chain: saveEncrypted() passes file to next handler Facade: Ciphers hides binary I/O from main()

This complete UML class diagram shows the full CipherCascade system. The abstract Ciphers class sits at the top of the hierarchy. All four concrete cipher classes inherit from it via Generalisation (——▷). Visibility is marked with - (private), # (protected), and + (public). The {abstract} tag marks the two pure virtual methods that every subclass must implement.